

INC-002 - High CPU Utilization / Auto Scaling

Terraform AWS High Availability Platform | Incident Testing and Recovery

Incident: High CPU utilization was deliberately generated on both EC2 instances to validate CloudWatch monitoring and automatic Auto Scaling scale-out and scale-in.

1. Environment

Component	Configuration
Cloud	AWS - ap-southeast-2 (Sydney)
Compute	Amazon EC2 - Amazon Linux 2023
Web Service	Apache HTTP Server (httpd)
Auto Scaling	project2-web-asg
ASG Capacity	Minimum 2 Desired 2 Maximum 4
Scaling Policy	Target Tracking - ASGAverageCPUUtilization
Target CPU	70%
Monitoring	Amazon CloudWatch
Access Method	AWS Systems Manager Session Manager

2. What Happened

The platform was running two healthy EC2 instances. CloudWatch showed a baseline CPU utilization of approximately 0.4% on each instance. CPU load was then deliberately generated on both instances through Session Manager.

Two background CPU-load processes were started on each instance. Both original instances reached approximately 100% CPU. The target-tracking high alarm entered ALARM and the Auto Scaling Group increased desired capacity from 2 to 3 and then from 3 to 4.

After the artificial load was stopped, CPU utilization fell. The target-tracking low alarm initiated scale-in, reducing desired capacity from 4 to 3 and eventually back to the configured minimum of 2. The final two instances were InService and Healthy.

3. Load Simulation

On each original EC2 instance, two CPU-intensive background processes were started:

```
yes > /dev/null &  
yes > /dev/null &
```

CPU activity was checked with:

```
top -bn1 | head -n 5
```

A reading of **0.0 id** showed that no CPU time was idle at that moment.

4. Detection and Troubleshooting

Baseline check: Confirmed minimum 2, desired 2, maximum 4, with two InService / Healthy instances.

CPU stress: Generated high CPU load on both EC2 instances.

Target-tracking detection: Observed TargetTracking AlarmHigh enter ALARM as ASG average CPU exceeded the 70% target.

Scale-out: Desired capacity increased 2 -> 3 and then 3 -> 4, with new EC2 instances launched automatically.

Load removal: Stopped the artificial load using `pkill -x yes`. On the reconnected instance, `pgrep -a yes` returned no output, confirming the load process was gone.

Scale-in: TargetTracking AlarmLow later reduced desired capacity 4 -> 3 -> 2.

Recovery validation: Confirmed the final two EC2 instances were InService and Healthy.

5. Key Diagnostic Evidence

Check	Observed Result
ASG before test	2 instances - InService / Healthy
Baseline CPU	Approximately 0.4% on both instances
CPU stress	Both original instances reached approximately 100% CPU
AlarmHigh	Entered ALARM
Scale-out	Desired capacity 2 -> 3 -> 4
Load stopped	yes processes terminated
Scale-in	Desired capacity 4 -> 3 -> 2
Final ASG state	2 instances - InService / Healthy

6. Resolution

No manual capacity change was required. After CPU load was removed, the target-tracking policy automatically reduced excess capacity and returned the group to two healthy instances.

7. Recovery Flow

```
Baseline: 2 healthy EC2 instances
|
High CPU generated on both instances
|
ASG average CPU exceeds 70% target
|
TargetTracking AlarmHigh enters ALARM
|
Desired capacity 2 -> 3 -> 4
|
Additional EC2 instances launched
|
Artificial CPU load stopped
|
Average CPU falls
|
TargetTracking AlarmLow triggers scale-in
|
Desired capacity 4 -> 3 -> 2
|
Final 2 instances InService / Healthy
```

8. Incident Outcome

Validation	Result
Low-CPU baseline confirmed	PASS
High CPU successfully generated	PASS
Target-tracking high alarm detected load	PASS
ASG automatically scaled 2 -> 3 -> 4	PASS
CPU load successfully stopped	PASS
Target-tracking low alarm initiated scale-in	PASS
ASG automatically returned 4 -> 3 -> 2	PASS
Final instances InService and Healthy	PASS

9. Lessons Learned

This test demonstrated how CloudWatch metrics and an Auto Scaling target-tracking policy work together. CloudWatch measured CPU utilization, while the Auto Scaling Group changed EC2 capacity to keep average CPU near the configured 70% target.

The test also showed why high CPU on only one of two instances may not be enough to exceed a 70% ASG average. Sustained load on both original instances raised the group average sufficiently to trigger scale-out. After demand disappeared, Auto Scaling removed excess capacity without going below the minimum size.

It also confirmed that the Application Load Balancer does not create EC2 instances. The Auto Scaling Group launches and terminates instances as capacity changes.

Suggested GitHub evidence: include the CloudWatch baseline graph, 100% CPU terminal evidence, TargetTracking AlarmHigh in ALARM, ASG activity showing 2 -> 3 -> 4, scale-in activity showing 4 -> 3 -> 2, and the final Instance Management view showing two InService / Healthy instances.